МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №2

«Структури даних та функціональна декомпозиція»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав:

студент групи ФЕП-32

Горин Михайло

Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №2. Тема: «Структури даних та функціональна декомпозиція».

3. Мета роботи

Сформувати практичні навички професійного використання базових та розширених структур даних Python (`list`, `tuple`, `set`, `dict`, модуль `collections`), навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю. У ході роботи опановано comprehensions, генераторні вирази, `*args`, `**kwargs`, `lambda`, closures та decorators.

4. Номер індивідуального варіанта

Варіант №5 — «Аналіз каталогу автомобілів». Лабораторна робота є продовженням ЛР1: використано той самий пакет `car_catalog` і репозиторій https://github.com/MykhailoG212/Prof_python.

5. Постановка задачі

Необхідно створити модуль обробки структурованих даних про автомобілі (марка, модель, рік, ціна та пробіг), який приймає набір об'єктів, виконує фільтрацію, пошук, групування, сортування, агрегацію та формування статистики, використовує кілька типів колекцій Python, розділений на невеликі функції, містить функції вищого порядку, closure та decorator.

5.1. Вимоги варіанта №5

Згідно із завданням, потрібно реалізувати такі можливості та показати, де саме вони реалізовані:

Вимога варіанта	Реалізація
унікальні марки	`get_unique_brands()` — set comprehension
кількість автомобілів кожної марки	`count_cars_by_brand()` — `Counter`
середня ціна	`calculate_average_price()`, `calculate_variadic_average(*prices)`, `calculate_total_and_average_price()`
найдорожчий автомобіль	`find_most_expensive()`, `find_most_expensive_car()` — `max` з `lambda`

Вимога варіанта	Реалізація
автомобіль із найменшим пробігом	`find_lowest_mileage()` — `min` з `lambda`
сортування за ціною	`sort_cars_by_price()` — `sorted` з `lambda`-ключем
фільтрація за роком	`filter_by_year()`, `create_min_year_filter()` + `filter_cars()`
dict-index за ID	`build_cars_by_id_index()` — dict comprehension

5.2. Загальні вимоги до програми

Згідно з розділом 25 завдання, програма повинна містити:

№	Вимога	Де виконано
1	щонайменше один `list`	`catalog: list[Car]`, результати фільтрів
2	щонайменше один `tuple`	`extract_cars_brief_tuples()`, `calculate_total_and_average_price() -> tuple`
3	щонайменше один `set`	`get_unique_brands()`
4	щонайменше один `dict`	`build_cars_by_id_index()`, `group_cars_by_brand()`
5	структура з `collections`	`Counter`, `defaultdict`, `deque`
6–8	list / set / dict comprehension	розділ 12
9	щонайменше 6 функцій	понад 20 функцій, розділ 10
10	функція з `*args`	`calculate_variadic_average()`
11	функція з `**kwargs`	`create_car_record()`
12	хоча б одна `lambda`	`sort_cars_by_price()`, `find_most_expensive*()`
13	хоча б один closure	`create_price_range_filter()`, `create_min_year_filter()`
14	хоча б один decorator	`measure_time`, `repeat(count)`
15–19	сортування, фільтрація, групування, агрегація, пошук	`analytics.py`, `processors.py`, `services.py`
20	оцінювання складності ≥ 3 операцій	розділ 18

6. Структура проєкту

Проєкт розвиває структуру ЛР1 (src-layout): до наявних модулів `models.py`, `services.py` і `main.py` додано модулі `processors.py`,

analytics.py, decorators.py та benchmark.py, а також тести нової функціональності.

```
lab2code/
├── src/
│   └── car_catalog/
│       ├── __init__.py          # ознака пакета, версія
│       ├── models.py            # датаклас Car з валідацією (ЛР1)
│       ├── services.py          # пошук, фільтр за роком, min/max (ЛР1)
│       ├── processors.py        # set/dict comprehension, Counter, defaultdict
│       ├── analytics.py         # closure, lambda, *args, **kwargs, агрегація
│       ├── decorators.py        # measure_time, repeat(count)
│       ├── benchmark.py         # порівняння пошуку list та dict
│       └── main.py              # CLI-меню, демонстрація ЛР2, deque-історія
├── tests/
│   ├── test_services.py         # тести моделі та сервісів (ЛР1)
│   ├── test main.py             # тести інтерактивного введення (ЛР1)
│   └── test_processors_analytics.py   # тести ЛР2
├── .gitignore
├── pyproject.toml
└── README.md
```

Рекомендована в завданні структура (data.py, processors.py, analytics.py, decorators.py) адаптована під варіант: роль data.py виконують датаклас Car у models.py та функція get_demo_cars() у main.py, а пакет названо car_catalog для збереження сумісності з ЛР1.

7. Опис структури даних

Вхідний набір — список незмінних об’єктів Car (@dataclass(slots=True, frozen=True)). Кожен автомобіль має поля brand, model, year, price, mileage; коректність значень перевіряє __post_init__ (непорожні марка й модель, year >= 1886, скінченна price >= 0, mileage >= 0). Властивість full_name повертає «марка модель». Демонстраційний каталог складається з чотирьох автомобілів:

Марка	Модель	Рік	Ціна, $	Пробіг, км
Tesla	Model Y Performance	2023	50 000	12 000
Toyota	Camry	2020	25 000	45 000
Honda	Civic	2021	22 000	30 000
Toyota	Corolla	2019	18 000	60 000

8. Пояснення вибору list, tuple, set, dict

Структура	Де використано	Чому саме вона
list	каталог list[Car]; результати filter_cars(), sort_cars_by_price()	потрібна впорядкована змінювана колекція: автомобілі додаються через меню, порядок важливий для виводу та рейтингу

Структура	Де використано	Чому саме вона
`tuple`	`extract_cars_brief_tuples()` → `(brand, model, year)`; результат `(total, average)`	короткий запис із кількох логічно пов'язаних значень, які не повинні змінюватися; розпаковується одним присвоєнням
`set`	`get_unique_brands()`	потрібні лише унікальні марки; дублікати відкидаються автоматично, перевірка належності — O(1)
`dict`	`build_cars_by_id_index()`, `group_cars_by_brand()`	пошук за ключем (ID, марка) без перебору всього списку
`Counter`	`count_cars_by_brand()`	готовий інструмент підрахунку кількості входжень
`defaultdict`	`group_cars_by_brand()`	групування без перевірки «чи є вже такий ключ»
`deque`	історія операцій у `run_menu()`	двостороння черга з `maxlen=3` автоматично зберігає лише останні операції

9. Приклад використання collections

Модуль `processors.py` використовує `Counter` та `defaultdict`, а `main.py` — `deque` з обмеженою довжиною:

```
from collections import Counter, defaultdict, deque


# processors.py
def count_cars_by_brand(cars):
    return Counter(car.brand for car in cars)


def group_cars_by_brand(cars):
    grouped = defaultdict(list)
    for car in cars:
        grouped[car.brand].append(car)
    return dict(grouped)


# main.py
history: deque[str] = deque(maxlen=3)
history.append("Переглянуто каталог")
```

Для демонстраційного каталогу `Counter` повертає `{'Tesla': 1, 'Toyota': 2, 'Honda': 1}`, а `defaultdict` формує три групи за марками. `deque(maxlen=3)` після четвертої операції автоматично видаляє найстаріший запис, тому пам'ять під історію обмежена.

10. Основні функції

Програму розділено на невеликі функції, кожна з яких виконує одну логічну задачу:

Функція	Модуль	Призначення
`find_by_brand`	services	пошук за маркою без урахування регістру
`filter_by_year`	services	фільтрація за роком випуску
`find_most_expensive`	services	найдорожчий автомобіль (`max`, `default=None`)
`find_lowest_mileage`	services	автомобіль з найменшим пробігом
`calculate_average_price`	services	середня ціна каталогу
`get_unique_brands`	processors	множина унікальних марок
`build_cars_by_id_index`	processors	dict-індекс за ID
`count_cars_by_brand`	processors	підрахунок авто за марками (`Counter`)
`group_cars_by_brand`	processors	групування за маркою (`defaultdict`)
`extract_cars_brief_tuples`	processors	короткі записи-кортежі
`create_price_range_filter`	analytics	closure: предикат діапазону цін
`create_min_year_filter`	analytics	closure: предикат мінімального року
`filter_cars`	analytics	універсальна фільтрація (функція вищого порядку)
`sort_cars_by_price`	analytics	сортування за ціною (`lambda`)
`calculate_variadic_average`	analytics	середнє довільної кількості цін (`*args`)
`create_car_record`	analytics	створення авто з іменованих полів (`**kwargs`)
`find_most_expensive_car`	analytics	максимум через `lambda`
`calculate_total_and_average_price`	analytics	сума та середнє (generator expression, `@measure_time`)
`measure_time`, `repeat`	decorators	декоратори вимірювання часу та повтору
`generate_dataset`, `benchmark_lookup`, `run_benchmark`	benchmark	генерація даних і порівняння пошуку
`run_lab2_demo`, `run_menu`	main	демонстрація можливостей і меню

11. Текст основних фрагментів коду

11.1. Файл src/car_catalog/processors.py

```
"""Обробка колекцій даних про автомобілі."""


from collections import Counter, defaultdict


from car_catalog.models import Car



def get_unique_brands(cars: list[Car]) -> set[str]:
    """Повертає множину марок автомобілів за допомогою set comprehension."""
    return {car.brand for car in cars}



def build_cars_by_id_index(cars: list[Car]) -> dict[int, Car]:
    """Будує індекс автомобілів за 1-based порядковим ID у каталозі."""
    return {car_id: car for car_id, car in enumerate(cars, start=1)}



def count_cars_by_brand(cars: list[Car]) -> Counter[str]:
    """Підраховує кількість автомобілів кожної марки."""
    return Counter(car.brand for car in cars)



def group_cars_by_brand(cars: list[Car]) -> dict[str, list[Car]]:
    """Групує автомобілі за маркою, використовуючи defaultdict."""
    grouped: defaultdict[str, list[Car]] = defaultdict(list)
    for car in cars:
        grouped[car.brand].append(car)
    return dict(grouped)



def extract_cars_brief_tuples(cars: list[Car]) -> list[tuple[str, str, int]]:
    """Повертає короткі незмінні записи (марка, модель, рік)."""
    return [(car.brand, car.model, car.year) for car in cars]
```

11.2. Файл src/car_catalog/analytics.py

```
"""Функціональні фільтри та агрегування даних каталогу."""


from collections.abc import Callable
from typing import Any


from car_catalog.decorators import measure_time
from car_catalog.models import Car



def create_price_range_filter(
    min_price: float,
    max_price: float,
) -> Callable[[Car], bool]:
    """Створює closure-предикат для фільтрації за включним діапазоном цін."""
    if min_price > max_price:
        raise ValueError("Мінімальна ціна не може перевищувати максимальну.")


    def predicate(car: Car) -> bool:
```

```
        return min_price <= car.price <= max_price

    return predicate



def create_min_year_filter(min_year: int) -> Callable[[Car], bool]:
    """Створює closure-предикат для фільтрації за мінімальним роком."""

    def predicate(car: Car) -> bool:
        return car.year >= min_year

    return predicate



def filter_cars(
    cars: list[Car],
    predicate: Callable[[Car], bool],
) -> list[Car]:
    """Фільтрує автомобілі за переданою функцією-предикатом."""
    return [car for car in cars if predicate(car)]



def sort_cars_by_price(
    cars: list[Car],
    descending: bool = False,
) -> list[Car]:
    """Повертає автомобілі, відсортовані за ціною lambda-ключем."""
    return sorted(cars, key=lambda car: car.price, reverse=descending)



def calculate_variadic_average(*prices: float) -> float:
    """Обчислює середнє значення довільної кількості цін."""
    if not prices:
        return 0.0
    return sum(prices) / len(prices)



def create_car_record(**fields: Any) -> Car:
    """Створює автомобіль із keyword-полів; ``make`` підтримується як alias."""
    allowed_fields = {"brand", "make", "model", "year", "price", "mileage"}
    unexpected_fields = fields.keys() - allowed_fields
    if unexpected_fields:
        names = ", ".join(sorted(unexpected_fields))
        raise TypeError(f"Невідомі поля автомобіля: {names}")
    if "brand" in fields and "make" in fields:
        raise TypeError("Вкажіть лише одне з полів: brand або make.")

    return Car(
        brand=str(fields.get("brand", fields.get("make", "Unknown"))),
        model=str(fields.get("model", "Unknown")),
        year=int(fields.get("year", 2000)),
        price=float(fields.get("price", 0.0)),
        mileage=int(fields.get("mileage", 0)),
    )



def find_most_expensive_car(cars: list[Car]) -> Car | None:
    """Повертає найдорожчий автомобіль або None для порожнього списку."""
    return max(cars, key=lambda car: car.price, default=None)



@measure_time
def calculate total and average price(cars: list[Car]) -> tuple[float, float]:
    """Обчислює сумарну та середню ціну без створення проміжного списку."""
    if not cars:
```

```
        return 0.0, 0.0
    total = sum(car.price for car in cars)
    return total, total / len(cars)
```

11.3. Файл src/car_catalog/decorators.py

```
"""Декоратори для вимірювання часу та повторного виконання функцій."""


from collections.abc import Callable
from functools import wraps
from time import perf_counter
from typing import ParamSpec, TypeVar, cast


P = ParamSpec("P")
R = TypeVar("R")



def measure_time(func: Callable[P, R]) -> Callable[P, R]:
    """Вимірює та друкує тривалість виконання функції."""


    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start_time = perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed = perf counter() - start time
            print(f"[BENCHMARK] {func.__name__} виконано за {elapsed:.8f} с")


    return cast(Callable[P, R], wrapper)



def repeat(count: int) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Параметризований декоратор, що виконує функцію задану кількість разів."""
    if count < 1:
        raise ValueError("Кількість повторень має бути не меншою за 1.")


    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            result = func(*args, **kwargs)
            for _ in range(count - 1):
                result = func(*args, **kwargs)
            return result


        return wrapper


    return decorator
```

11.4. Файл src/car_catalog/benchmark.py

```
"""Порівняння лінійного пошуку в list з пошуком за ключем у dict."""


from time import perf counter


from car catalog.models import Car



def generate dataset(size: int) -> list[Car]:
    """Генерує набір автомобілів із детермінованими, унікальними моделями."""
    if size < 1:
```

```
        raise ValueError("Розмір набору має бути додатним.")
    return [
        Car(
            brand=f"Brand_{index % 50}",
            model=f"Model_{index}",
            year=2000 + index % 25,
            price=10_000.0 + index % 50_000,
            mileage=index * 10,
        )
        for index in range(size)
    ]



def benchmark_lookup(size: int) -> tuple[float, float]:
    """Повертає час пошуку останнього запису через list та dict."""
    cars = generate_dataset(size)
    target_id = size
    target = cars[-1]
    records = list(enumerate(cars, start=1))
    index = {car_id: car for car_id, car in records}
    iterations = 100


    found_in_list = next(
        (car for car_id, car in records if car_id == target_id),
        None,
    )
    found_in_dict = index.get(target_id)
    if found_in_list is not target or found_in_dict is not target:
        raise RuntimeError("Пошук у list та dict повернув різні результати.")


    start = perf_counter()
    for _ in range(iterations):
        next(car for car_id, car in records if car_id == target_id)
    list_elapsed = (perf_counter() - start) / iterations


    start = perf_counter()
    for _ in range(iterations):
        index.get(target_id)
    dict_elapsed = (perf_counter() - start) / iterations
    return list_elapsed, dict_elapsed



def run_benchmark() -> None:
    """Виводить результати benchmark для 1 000, 10 000 і 100 000 записів."""
    sizes = (1_000, 10_000, 100_000)
    print("\nПорівняння пошуку останнього автомобіля (побудова індексу не вимірюється)")
    print(f"{'Записів':>12} | {'list O(n), с':>16} | {'dict O(1), с':>16}")
    print("-" * 52)
    for size in sizes:
        list_elapsed, dict_elapsed = benchmark_lookup(size)
        print(f"{size:>12,} | {list_elapsed:>16.8f} | {dict_elapsed:>16.8f}")
    print(
        "\nВисновок: лінійний пошук у списку перевіряє до n записів (O(n)); "
        "пошук за ключем у готовому словнику має середню складність O(1)."
    )



if __name__ == "__main__":
    run_benchmark()
```

11.5. Файл src/car_catalog/main.py (функція run_lab2_demo)

```
def run_lab2_demo(catalog: list[Car], history: deque[str]) -> None:
    unique_brands = get_unique_brands(catalog)
    print(f"\n[set comprehension] Унікальні марки: {unique_brands}")


    counts = count_cars_by_brand(catalog)
    print(f"[Counter] Кількість авто за марками: {dict(counts)}")
```

```
    grouped = group_cars_by_brand(catalog)
    print(f"[defaultdict] Кількість груп за марками: {len(grouped)}")


    car_index = build_cars_by_id_index(catalog)
    print(f"[dict comprehension] Автомобіль з ID 1: {car_index.get(1)}")


    brief_records = extract_cars_brief_tuples(catalog)
    print(f"[list comprehension + tuple] Перші записи: {brief_records[:3]}")


    price_filter = create_price_range_filter(20_000.0, 75_000.0)
    filtered_cars = filter_cars(catalog, price_filter)
    print(
        "[closure + higher-order filter] Авто за ціною $20,000-$75,000: "
        f"{[car.full_name for car in filtered_cars]}"
    )


    recent_cars = filter_cars(catalog, create_min_year_filter(2021))
    print(
        "[closure filter] Авто від 2021 року: "
        f"{[car.full_name for car in recent_cars]}"
    )


    sorted_cars = sort_cars_by_price(catalog, descending=True)
    print(
        "[lambda sort] Найдорожчі авто: "
        f"{[car.full_name for car in sorted_cars[:3]]}"
    )
    print(
        "[*args] Середня ціна: "
        f"${calculate_variadic_average(*(car.price for car in catalog)):,.2f}"
    )


    created_car = create_car_record(
        brand="Mercedes-Benz",
        model="AMG GT",
        year=2022,
        price=115_000.0,
        mileage=8_000,
    )
    print(f"[**kwargs] Створено запис: {created_car.full_name}")


    total, average = calculate_total_and_average_price(catalog)
    print(f"[generator aggregation] Загальна ціна: ${total:,.2f}; середня: ${average:,.2f}")
    most_expensive = find_most_expensive_car(catalog)
    if most_expensive is not None:
        print(f"[lambda aggregation] Найдорожчий автомобіль: {most_expensive.full_name}")


    history.append("Виконано демонстрацію ЛР2")
    print(f"[deque] Останні операції: {list(history)}")
```

Інтеграцію в меню виконано додаванням пунктів 9 і 10 та історії `deque` у функції `run_menu()`:

```
def run_menu(catalog: list[Car]) -> None:
    history: deque[str] = deque(maxlen=3)
    while True:
        ...
        elif choice == "9":
            run_lab2_demo(catalog, history)
        elif choice == "10":
            run_benchmark()
            history.append("Виконано benchmark list vs dict")
```

Файли `models.py` і `services.py` (пошук, фільтрація за роком, мінімум, максимум і середнє) використовуються з ЛР1 без змін.

12. Приклади comprehensions

Вид	Приклад із проєкту	Результат
list comprehension	`[(car.brand, car.model, car.year) for car in cars]`	список кортежів-записів
list comprehension з умовою	`[car for car in cars if predicate(car)]`	відфільтровані автомобілі
set comprehension	`{car.brand for car in cars}`	множина унікальних марок
dict comprehension	`{car_id: car for car_id, car in enumerate(cars, start=1)}`	індекс «ID → автомобіль»
generator expression	`sum(car.price for car in cars)`	сума без проміжного списку

Generator expression у `calculate_total_and_average_price()` не створює проміжного списку цін: значення обчислюються по одному під час підсумовування, що економить пам’ять на великих каталогах.

13. Приклад *args

Функція приймає довільну кількість цін і повертає їх середнє; для порожнього виклику повертається `0.0`:

```
def calculate_variadic_average(*prices: float) -> float:
    """Обчислює середнє значення довільної кількості цін."""
    if not prices:
        return 0.0
    return sum(prices) / len(prices)
```

Виклик у `main.py` розпаковує генератор цін: `calculate_variadic_average(*(car.price for car in catalog))` → $28 750,00.

14. Приклад **kwargs

Функція отримує довільні keyword-аргументи, відхиляє невідомі поля (`TypeError`), підтримує синонім `make` для `brand` та підставляє значення за замовчуванням:

```
def create_car_record(**fields: Any) -> Car:
    """Створює автомобіль із keyword-полів; ``make`` підтримується як alias."""
    allowed_fields = {"brand", "make", "model", "year", "price", "mileage"}
    unexpected_fields = fields.keys() - allowed_fields
    if unexpected_fields:
```

```
        names = ", ".join(sorted(unexpected_fields))
        raise TypeError(f"Невідомі поля автомобіля: {names}")
    if "brand" in fields and "make" in fields:
        raise TypeError("Вкажіть лише одне з полів: brand або make.")


    return Car(
        brand=str(fields.get("brand", fields.get("make", "Unknown"))),
        model=str(fields.get("model", "Unknown")),
        year=int(fields.get("year", 2000)),
        price=float(fields.get("price", 0.0)),
        mileage=int(fields.get("mileage", 0)),
    )
```

Виклик: `create_car_record(brand="Mercedes-Benz", model="AMG GT", year=2022, price=115_000.0, mileage=8_000)`.

15. Приклад lambda

Анонімні функції використано як короткі ключі сортування та пошуку екстремумів:

```
sorted(cars, key=lambda car: car.price, reverse=descending)   # analytics.py
max(cars, key=lambda car: car.price, default=None)            # services.py, analytics.py
min(cars, key=lambda car: car.mileage, default=None)          # services.py
```

Складну бізнес-логіку у `lambda` не винесено: вона розміщена в іменованих функціях із docstring, які можна окремо тестувати.

16. Пояснення closure

Closure — це внутрішня функція, що зберігає доступ до змінних зовнішньої функції після її завершення. У `create_price_range_filter(min_price, max_price)` внутрішня функція `predicate` «запам’ятовує» межі діапазону:

```
def create_price_range_filter(
    min_price: float,
    max_price: float,
) -> Callable[[Car], bool]:
    """Створює closure-предикат для фільтрації за включним діапазоном цін."""
    if min_price > max_price:
        raise ValueError("Мінімальна ціна не може перевищувати максимальну.")


    def predicate(car: Car) -> bool:
        return min price <= car.price <= max price


    return predicate
```

Фабрика викликається один раз (`create_price_range_filter(20_000.0, 75_000.0)`), а отриманий предикат передається в `filter_cars()` як функція вищого порядку й застосовується до кожного автомобіля. Так одна універсальна функція

фільтрації обслуговує різні критерії без дублювання коду. Аналогічно працює `create_min_year_filter(min_year)`.

17. Пояснення decorator

Decorator додає поведінку функції без зміни її коду. `measure_time` вимірює час виконання за допомогою `perf_counter()` і друкує його навіть у разі винятку (`try/finally`), а `functools.wraps` зберігає ім'я, docstring і сигнатуру оригіналу. Він застосований до `calculate_total_and_average_price()`:

```
@measure_time
def calculate_total_and_average_price(cars: list[Car]) -> tuple[float, float]:
    ...
```

Параметризований decorator `repeat(count)` — це одночасно closure (`count` зберігається у вкладених функціях), decorator, `*args` та `**kwargs`: зовнішня функція приймає параметр, середня — функцію, внутрішня `wrapper` виконує її задану кількість разів і повертає останній результат. Його перевіряє тест `test_repeat_runs_function_and_preserves_metadata`.

18. Результат роботи програми

Нижче наведено вивід реального запуску проєкту: демонстрація можливостей ЛР2 (пункт меню 9), benchmark (пункт 10) та результат тестів.

Рисунок 1 — Демонстрація можливостей ЛР2 (пункт меню 9)

```
[set comprehension] Унікальні марки: {'Toyota', 'Honda', 'Tesla'}
[Counter] Кількість авто за марками: {'Tesla': 1, 'Toyota': 2, 'Honda': 1}
[defaultdict] Кількість груп за марками: 3
[dict comprehension] Автомобіль з ID 1: Car(brand='Tesla', model='Model Y Performance',
year=2023, price=50000.0, mileage=12000)
[list comprehension + tuple] Перші записи: [('Tesla', 'Model Y Performance', 2023), ('Toyota',
'Camry', 2020), ('Honda', 'Civic', 2021)]
[closure + higher-order filter] Авто за ціною $20,000-$75,000: ['Tesla Model Y Performance',
'Toyota Camry', 'Honda Civic']
[closure filter] Авто від 2021 року: ['Tesla Model Y Performance', 'Honda Civic']
[lambda sort] Найдорожчі авто: ['Tesla Model Y Performance', 'Toyota Camry', 'Honda Civic']
[*args] Середня ціна: $28,750.00
[**kwargs] Створено запис: Mercedes-Benz AMG GT
[BENCHMARK] calculate total and average price виконано за 0.00000252 с
[generator aggregation] Загальна ціна: $115,000.00; середня: $28,750.00
[lambda aggregation] Найдорожчий автомобіль: Tesla Model Y Performance
[deque] Останні операції: ['Виконано демонстрацію ЛР2']
```

Рисунок 1

Рисунок 2 — Результат benchmark (пункт меню 10)

```
Порівняння пошуку останнього автомобіля (побудова індексу не вимірюється)
     Записів |     list O(n), с |     dict O(1), с
```

```
------------------------------------------------
       1,000 |       0.00002044 |       0.00000004
      10,000 |       0.00020905 |       0.00000006
     100,000 |       0.00204370 |       0.00000008


Висновок: лінійний пошук у списку перевіряє до n записів (O(n)); пошук за ключем у готовому
словнику має середню складність O(1).
```

Рисунок 2

Рисунок 3 — Результат запуску pytest

```
============================= test session starts ==============================
platform linux -- Python 3.13, pytest-9.1.1, pluggy-1.6.0
rootdir: lab2code
configfile: pyproject.toml
testpaths: tests
collecting ... collected 19 items


tests/test_main.py::test_create_car_from_input_retries_invalid_values PASSED [  5%]
tests/test_main.py::test_menu_adds_car_to_catalog PASSED                 [ 10%]
tests/test_processors_analytics.py::test_processors_cover_collection_operations PASSED [ 15%]
tests/test_processors_analytics.py::test_processors_handle_empty_catalog PASSED [ 21%]
tests/test_processors_analytics.py::test_closure_filters_and_universal_filter PASSED [ 26%]
tests/test_processors_analytics.py::test_sorting_and_aggregations PASSED [ 31%]
tests/test_processors_analytics.py::test_create_car_record_accepts_brand_and_make_alias PASSED [ 36%]
tests/test_processors_analytics.py::test_create_car_record_rejects_ambiguous_or_unknown_fields PASSED [ 42%]
tests/test_processors_analytics.py::test_repeat_runs_function_and_preserves_metadata PASSED [ 47%]
tests/test_processors_analytics.py::test_measure_time_preserves_return_and_reports_elapsed PASSED [ 52%]
tests/test_processors_analytics.py::test_benchmark_lookup_and_dataset_validation PASSED [ 57%]
tests/test_processors_analytics.py::test_lab2_demo_records_history PASSED [ 63%]
tests/test_services.py::test_filter_by_make PASSED                       [ 68%]
tests/test_services.py::test_filter_by_year PASSED                       [ 73%]
tests/test_services.py::test_find_most_expensive_car PASSED              [ 78%]
tests/test_services.py::test_find_lowest_mileage_car PASSED              [ 84%]
tests/test_services.py::test_calculate_average_price PASSED              [ 89%]
tests/test_services.py::test_car_validation PASSED                       [ 94%]
tests/test_services.py::test_empty_catalog_results PASSED                [100%]


============================== 19 passed in 0.05s ==============================
```

Рисунок 3

Усі 19 тестів успішні: 9 тестів ЛР1 (модель, сервіси, інтерактивне введення) і 10 нових тестів ЛР2, що охоплюють колекції, closure-фільтри, сортування, агрегації, `*args`, `**kwargs`, обидва декоратори, benchmark та демонстраційний режим.

19. Аналіз складності операцій

19.1. Теоретична складність структур

Операція	list	set	dict
доступ за індексом	O(1)	—	—
пошук значення	O(n)	O(1) середнє	O(1) середнє (за ключем)
append	O(1) амортизовано	—	—
insert на початок	O(n)	—	—
додавання	—	O(1) середнє	O(1) середнє
видалення	O(n)	O(1) середнє	O(1) середнє

Операція	list	set	dict
iteration	O(n)	O(n)	O(n)

19.2. Складність операцій проєкту

Операція	Реалізація	Складність
Пошук автомобіля за ID у `list`	перебір елементів	O(n)
Побудова dict-індексу	dict comprehension	O(n)
Пошук за ID у готовому `dict`	`index.get(car_id)`	O(1) середнє
Унікальні марки	set comprehension	O(n)
Перевірка марки в `set`	`brand in brands`	O(1) середнє
Підрахунок за марками	`Counter`	O(n)
Групування за маркою	`defaultdict`	O(n)
Фільтрація за роком / ціною	list comprehension + предикат	O(n)
Найдорожчий / найменший пробіг	`max` / `min` з `lambda`	O(n)
Середня ціна	`sum` + generator expression	O(n)
Сортування за ціною	`sorted` (Timsort)	O(n log n)
Додавання в `deque`	`append` із `maxlen`	O(1)

20. Результати benchmark

Експеримент порівнює пошук останнього автомобіля (найгірший випадок для лінійного пошуку) у `list` та у `dict` для 1 000, 10 000 та 100 000 записів. Набір даних генерує `generate_dataset()`; кожен вимір усереднено за 100 ітерацій, а побудова індексу в час пошуку не входить. Перед вимірюванням перевіряється, що обидва способи повертають один і той самий об’єкт.

Кількість records	list search, с	dict search, с	Різниця, разів
1 000	0.00002044	0.00000004	≈ 511
10 000	0.00020905	0.00000006	≈ 3 484
100 000	0.00204370	0.00000008	≈ 25 546

Результати узгоджуються з теорією: час лінійного пошуку в `list` зростає приблизно пропорційно кількості записів (збільшення даних у 10 разів збільшує час приблизно в 10 разів — O(n)), тоді як час пошуку в `dict`

практично не залежить від розміру набору й залишається на рівні десятків–сотень наносекунд (O(1) в середньому). Чим більший набір даних, тим більша перевага словника. Точні значення залежать від комп'ютера й навантаження, але характер залежності відтворюється при кожному запуску. Ціна швидкого пошуку — одноразова побудова індексу за O(n) і додаткова пам'ять, тому dict-індекс виправданий, коли пошуків багато.

21. Завдання підвищеної складності

Реалізовано шість із десяти додаткових можливостей:

Можливість	**Реалізація**
універсальна функція фільтрації	`filter_cars(cars, predicate)`
параметризований decorator	`repeat(count)`
closure для створення фільтрів	`create_price_range_filter()`, `create_min_year_filter()`
статистика через `Counter`	`count_cars_by_brand()`
`deque` для історії останніх операцій	`history = deque(maxlen=3)` у `run_menu()`
benchmark list vs dict	модуль `benchmark.py`, пункт меню 10

22. Відповіді на контрольні запитання

1. Що таке list?

Впорядкована змінювана колекція з доступом за індексом; підтримує `append`, `remove`, `sort`. У проєкті каталог — це `list[Car]`.

2. Чим list відрізняється від tuple?

`list` можна змінювати після створення, `tuple` — ні. Кортеж легший за пам'яттю і може бути ключем `dict`, якщо містить hashable-елементи.

3. Чому tuple називається immutable?

Після створення не можна змінити склад кортежу чи присвоїти новий елемент за індексом; будь-яка «зміна» створює новий об'єкт.

4. Для чого використовується set?

Для зберігання унікальних елементів, швидкої перевірки належності та теоретико-множинних операцій (`|`, `&`, `-`, `^`). У проєкті — унікальні марки.

5. Яка властивість елементів set є важливою?

Елементи мають бути hashable (мати стабільний хеш і `__eq__`); дублікати відкидаються, порядок не гарантується.

6. Що таке dict?

Колекція пар «ключ → значення» з швидким доступом за ключем. У проєкті — індекс автомобілів за ID та групи за марками.

7. Що таке key-value structure?

Структура, у якій кожному унікальному ключу відповідає значення; доступ відбувається за ключем, а не за позицією.

8. Чому пошук key у dict зазвичай швидкий?

`dict` — хеш-таблиця: за `hash(key)` одразу обчислюється позиція запису, тому перебирати всі елементи не потрібно.

9. Що таке hashability?

Здатність об'єкта мати незмінний протягом життя хеш (`__hash__`) і порівнюватися (`__eq__`). Це умова для ключів `dict` та елементів `set`. Frozen-датаклас `Car` hashable.

10. Чому list не може бути key у dict?

Список змінюваний, тому його вміст і хеш могли б змінитися; Python забороняє це (`TypeError: unhashable type: 'list'`).

11. Для чого використовується Counter?

Для підрахунку кількості входжень елементів. `count_cars_by_brand()` повертає `Counter` марок.

12. Для чого використовується defaultdict?

Автоматично створює значення за замовчуванням для відсутнього ключа, що спрощує групування: `defaultdict(list)` у `group_cars_by_brand()`.

13. Для чого використовується deque?

Двостороння черга з додаванням і видаленням з обох кінців за O(1); з `maxlen` автоматично відкидає найстаріші елементи (історія операцій).

14. Що таке list comprehension?

Компактний запис створення списку: `[вираз for елемент in послідовність if умова]`, наприклад у `filter_cars()`.

15. Що таке set comprehension?

Аналогічний запис для множини: `{car.brand for car in cars}`.

16. Що таке dict comprehension?

Запис для словника: `{car_id: car for car_id, car in enumerate(cars, start=1)}`.

17. Що таке generator expression?

Вираз у круглих дужках, що обчислює елементи ліниво, по одному, без створення проміжного списку: `sum(car.price for car in cars)`.

18. Що означає O(1)?

Постійний час: тривалість операції не залежить від кількості даних (пошук у `dict` за ключем у середньому).

19. Що означає O(n)?

Лінійний час: тривалість зростає пропорційно кількості елементів (перебір списку, фільтрація, `sum`).

20. Що означає O(n log n)?

Квазілінійний час, типовий для ефективного сортування (`sorted`, Timsort).

21. Що означає O(n²)?

Квадратичний час: тривалість зростає як квадрат розміру даних, типово для вкладених циклів по всіх парах елементів; у проєкті такої складності немає.

22. Яка приблизна складність пошуку у list?

O(n) — у гіршому випадку доведеться переглянути всі елементи.

23. Яка середня складність пошуку у set?

O(1) — перевірка `x in set` за хешем.

24. Яка середня складність пошуку key у dict?

O(1) в середньому (у найгіршому випадку при масових колізіях — O(n)).

25. Чому правильна структура даних може прискорити програму?

Вона знижує асимптотичну складність: dict-індекс замість перебору списку в benchmark прискорює пошук у тисячі й більше разів на великих наборах.

26. Що таке функціональна декомпозиція?

Розбиття програми на невеликі функції, кожна з яких виконує одну чітко визначену задачу.

27. Які переваги малих functions?

Читабельність, повторне використання, простіше тестування та налагодження, локалізація помилок, легше розширювати.

28. Що таке positional argument?

Аргумент, значення якого визначається позицією у виклику: `filter_by_year(cars, 2021)`.

29. Що таке keyword argument?

Аргумент, переданий за іменем: `sort_cars_by_price(cars, descending=True)`.

30. Для чого використовується *args?

Збирає довільну кількість позиційних аргументів у `tuple`: `calculate_variadic_average(*prices)`.

31. Для чого використовується **kwargs?

Збирає довільні іменовані аргументи у `dict`: `create_car_record(**fields)`.

32. Що таке lambda function?

Коротка анонімна функція-вираз: `lambda car: car.price`.

33. Де доцільно використовувати lambda?

Як короткий ключ для `sorted`, `max`, `min` та інших функцій вищого порядку.

34. Чому складну business logic не варто записувати в lambda?

Вона втрачає читабельність, не має імені й docstring, її важко тестувати та налагоджувати; для цього краще використовувати `def`.

35. Що таке closure?

Внутрішня функція, що зберігає доступ до змінних зовнішньої функції після її завершення (`predicate` у `create_price_range_filter`).

36. Які variables може пам'ятати closure?

Вільні змінні з охоплювальної області видимості (наприклад, `min_price`, `max_price`); вони зберігаються в `__closure__`, а для зміни їх значень потрібен `nonlocal`.

37. Що таке decorator?

Функція, що приймає іншу функцію та повертає нову з додатковою поведінкою (`@measure_time`, `@repeat(3)`).

38. Для чого застосовуються decorators?

Для логування, вимірювання часу, повторів, кешування, перевірки доступу — без зміни коду самої функції.

39. Для чого використовується functools.wraps?

Копіює ім’я, docstring і сигнатуру оригінальної функції у wrapper; це перевіряє тест із `inspect.signature`.

40. Чому потрібно оцінювати складність алгоритму?

Щоб передбачити поведінку на великих даних, свідомо обрати структуру й уникнути вузьких місць до того, як вони стануть проблемою.

23. Висновок

У лабораторній роботі №2 до каталогу автомобілів (варіант №5) додано модуль обробки структурованих даних. Для різних задач обрано відповідні структури: `list` для впорядкованого каталогу, `tuple` для незмінних коротких записів, `set` для унікальних марок, `dict` для індексу за ID, а також `Counter`, `defaultdict` і `deque` з модуля `collections`.

Застосовано list/set/dict comprehensions і generator expressions, розділено логіку на невеликі функції, реалізовано `*args`, `**kwargs`, `lambda`, closure-фільтри та два декоратори — `measure_time` і параметризований `repeat`.

Реалізовано сортування, фільтрацію, групування, агрегацію, пошук і формування статистики.

Аналіз складності та benchmark підтвердили теорію: пошук у `list` має складність O(n), а в готовому `dict` — в середньому O(1), і з ростом набору даних від 1 000 до 100 000 записів різниця стрімко зростає. Код покрито 19 автоматичними тестами `pytest`, усі вони успішні. Результатом роботи є окремий модуль обробки структурованих даних, який можна використовувати як компонент наскрізного навчального проєкту.